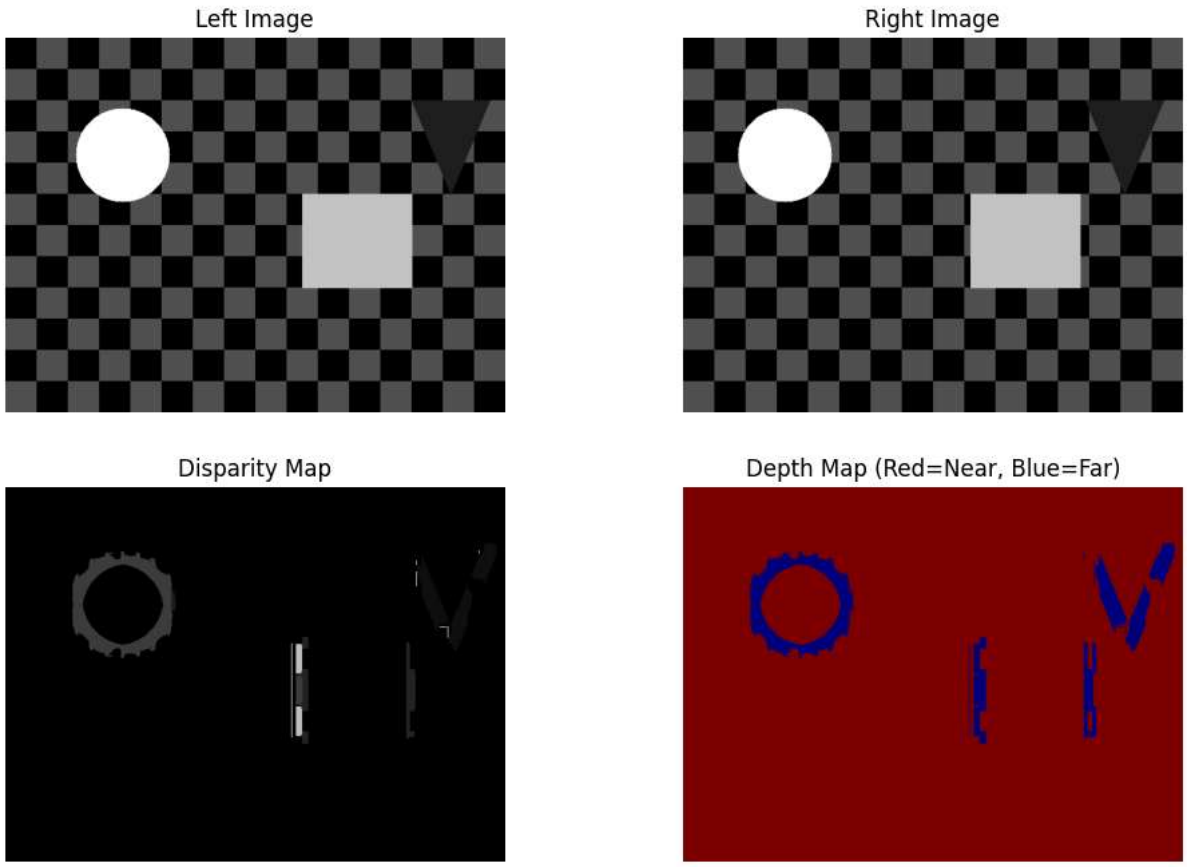
```
# ----------------------------------------------------------
# IMPORTS
# ----------------------------------------------------------
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------------------------------------
# SYNTHETIC STEREO IMAGE GENERATION
# ----------------------------------------------------------
def generate_stereo_images():
    print("Generating synthetic stereo images...\n")

    height, width = 480, 640
    left_img = np.ones((height, width), dtype=np.uint8) * 150

    # Checkerboard background
    square_size = 40
    for i in range(0, height, square_size):
        for j in range(0, width, square_size):
            if (i // square_size + j // square_size) % 2 == 0:
                left_img[i:i+square_size, j:j+square_size] = 100

    # Objects (different depths)
    cv2.circle(left_img, (150, 150), 60, 255, -1)   # close
    cv2.rectangle(left_img, (380, 200), (520, 320), 220, -1)  # mid

    pts = np.array([[520, 80], [620, 80], [570, 200]], np.int32)
    cv2.fillPoly(left_img, [pts], 120)  # far

    # Right image (shifted)
    right_img = np.ones((height, width), dtype=np.uint8) * 150

    for i in range(0, height, square_size):
        for j in range(0, width, square_size):
            if (i // square_size + j // square_size) % 2 == 0:
                right_img[i:i+square_size, j:j+square_size] = 100

    # Shift objects (simulate depth)
    cv2.circle(right_img, (130, 150), 60, 255, -1)   # shift more → close
    cv2.rectangle(right_img, (368, 200), (508, 320), 220, -1)

    pts_r = np.array([[515, 80], [615, 80], [565, 200]], np.int32)
    cv2.fillPoly(right_img, [pts_r], 120)

    print("✓ Synthetic stereo images generated!\n")
    return left_img, right_img

# ----------------------------------------------------------
# DEPTH COMPUTATION
# ----------------------------------------------------------
def compute_depth(left_img, right_img):
    print("Computing disparity and depth...")

    stereo = cv2.StereoBM_create(numDisparities=16*5, blockSize=15)

    disparity = stereo.compute(left_img, right_img).astype(np.float32)

    # Avoid division issues
    disparity[disparity <= 0] = 0.1

    baseline = 120   # mm
    focal_length = 400  # pixels

    depth = (baseline * focal_length) / disparity

    print("✓ Depth computed!\n")
    return disparity, depth

# ----------------------------------------------------------
# VISUALIZATION
# ----------------------------------------------------------
def visualize_all(left_img, right_img, disparity, depth):

    disparity_norm = cv2.normalize(disparity, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
```

```
    depth_norm = cv2.normalize(depth, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

    depth_color = cv2.applyColorMap(depth_norm, cv2.COLORMAP_JET)
    depth_color = cv2.cvtColor(depth_color, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(12, 8))

    plt.subplot(2,2,1)
    plt.imshow(left_img, cmap='gray')
    plt.title("Left Image")
    plt.axis("off")

    plt.subplot(2,2,2)
    plt.imshow(right_img, cmap='gray')
    plt.title("Right Image")
    plt.axis("off")

    plt.subplot(2,2,3)
    plt.imshow(disparity_norm, cmap='gray')
    plt.title("Disparity Map")
    plt.axis("off")

    plt.subplot(2,2,4)
    plt.imshow(depth_color)
    plt.title("Depth Map (Red=Near, Blue=Far)")
    plt.axis("off")

    plt.show()

    # Stats
    valid = depth[depth > 0]
    print("Min Depth:", np.min(valid))
    print("Max Depth:", np.max(valid))
    print("Mean Depth:", np.mean(valid))

# ----------------------------------------------------------
# MAIN
# ----------------------------------------------------------
def main():
    print("STEREO VISION & DEPTH ESTIMATION\n")

    left_img, right_img = generate_stereo_images()

    disparity, depth = compute_depth(left_img, right_img)

    visualize_all(left_img, right_img, disparity, depth)

    # Save outputs
    cv2.imwrite("left.png", left_img)
    cv2.imwrite("right.png", right_img)

    print("\n✓ Images saved!")

# ----------------------------------------------------------
# RUN
# ----------------------------------------------------------
main()
```

```
STEREO VISION & DEPTH ESTIMATION

Generating synthetic stereo images...

✓ Synthetic stereo images generated!

Computing disparity and depth...
✓ Depth computed!
```




```
Min Depth: 37.97468
Max Depth: 480000.0
Mean Depth: 459457.22

✓ Images saved!
```

Start coding or generate with AI.